Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. S. Crespi Reghizzi, prof. A. Morzenti)

Written exam - 28 June 2013 - Part I: Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY COMMENTED - THE CANDIDATE SHOULD CORRECTLY ANSWER AND REASONABLY EXPLAIN WHY

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME:

SURNAME:

ID: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the two regular expressions X and Y below:

$$X = (\, a^{+}\, b\,)^{*} \qquad\qquad Y = (\, a\, b\,)^{+}\; b^{+}$$

Answer the following questions:

(a) As simply as possible, build a finite automaton A (not necessarily deterministic) that recognizes the union language $L(X) \cup L(Y)$.

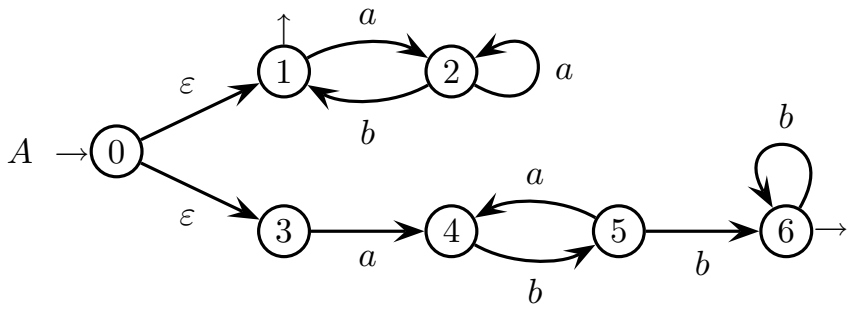
(b) Again for the union language $L(X) \cup L(Y)$, find a deterministic automaton A' by a method at choice.

(c) For the concatenation language $L(X) \cdot L(Y)$, write a regular expression Z, verify that it is not ambiguous, and if it is not so, then modify it so as to remove its ambiguity.

(d) (optional) Verify that the deterministic automaton A' found at point (b) is minimal, and if it is not so, minimize it.

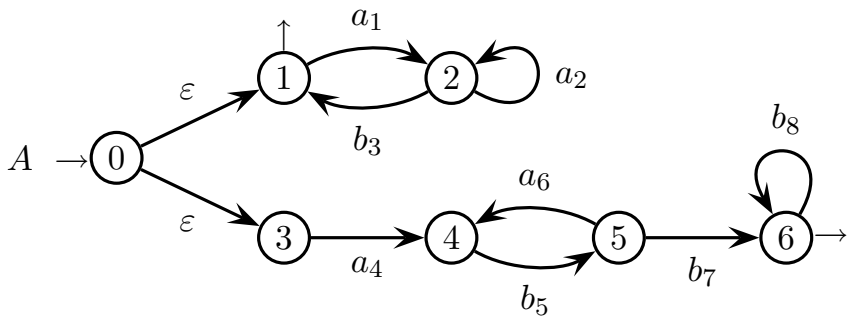
Solution

(a) Here is the automaton A that recognizes the union language $L(X) \cup L(Y)$:



The construction is easy: parts $1-2$ and $3-6$ recognize expressions X and Y, respectively, then united by spontaneous transitions from the initial state 0. Automaton A is nondeterministic, because of the spontaneous transitions it has.

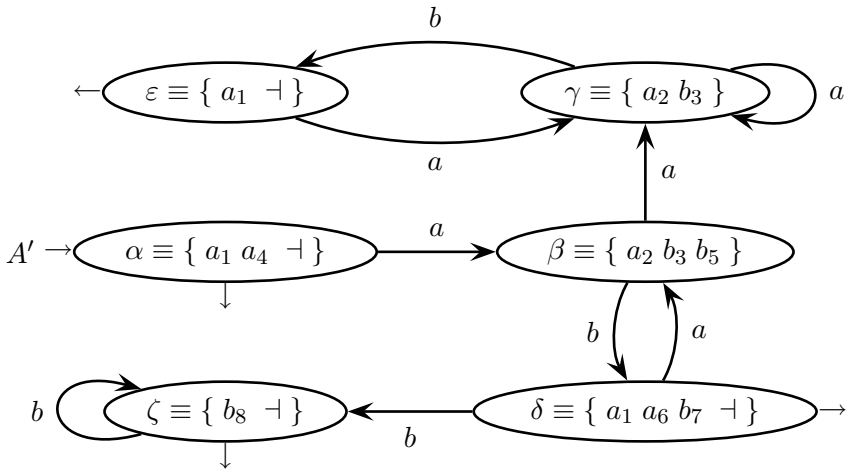
(b) Here is the minimal deterministic recognizer A' for the concatenation language $L(X) \cdot L(Y)$. To build it, we can make deterministic the previous automaton A. To this purpose, we use the Berry-Sethi method. First we number the arc labels (but of course not in the ε-transitions):



Then we compute the initials and the Follow sets:

initials	$a_1\ a_4\ \dashv$
generator	*follow set*
a_1	$a_2\ b_3$
a_2	$a_2\ b_3$
b_3	$a_1\ \dashv$
a_4	b_5
b_5	$a_6\ b_7$
a_6	b_5
b_7	$b_8\ \dashv$
b_8	$b_8\ \dashv$

Finally we construct the *DFA* A' equivalent to A:



This *DFA* A' has six states and is deterministic by construction.

(c) Here is the regular expression Z for the concatenation language $L(X) \cdot L(Y)$. The straightforward formulation below:

$$Z = X \cdot Y = \left(a^+ b\right)^* \cdot (a\,b)^+ \; b^+$$

is ambiguous, as witnessed by the two segmentations of the phrase:

$$\overbrace{a^2\,b}^{e_1}\;\overbrace{a\,b\,a\,b\,b}^{e_2} \qquad\qquad \overbrace{a^2\,b\,a\,b}^{e_1}\;\overbrace{a\,b\,b}^{e_2}$$

To obtain an unambiguous formula, we decompose the subexpression $(a\,b)^+$ of Y into $(a\,b)^*\,a\,b$, and we observe the following identity:

$$\begin{aligned} Z &= \left(a^+ b\right)^* \cdot (a\,b)^+ \; b^+ = \\ &= \left(a^+ b\right)^* \cdot (a\,b)^* \; a\,b\,b^+ = \\ &= \overbrace{\left(a^+ b\right)^*}^{e_1} \cdot \overbrace{a\,b\,b^+}^{e_3} \end{aligned}$$

because $(a^+\,b)^* \cdot (a\,b)^* = (a^+\,b)^*$, i.e., $(a\,b)^*$ can be "adsorbed" by $(a^+\,b)^*$. This r.e. Z is unambiguous: subexpression e_3, but not subexpression e_1, can span over a suffix of the form $a\,b\,\ldots\,b$; and every prefix $(a^+\,b)^*$, except the last occurrence of $a\,b$, can be spanned by e_1 but not by e_2.

(d) We check if the *DFA* A' is minimal by computing the undistinguishability relation between the states:

state	β	γ	δ	ε	ζ
α	×	×	×	~~$(\beta,\ \gamma)$~~	×
β		~~$(\delta,\ \varepsilon)$~~	×	×	×
γ			×	×	×
δ				×	×
ε					×

We see that all the states are distinguishable and that the *DFA* A' is minimal.

2 Free Grammars and Pushdown Automata 20%

1. Consider a subset of the Dyck language with three parenthesis types (round, square and curly), defined through the following clauses:

   (a) in the sequences of the parenthesis pairs (nests) at the same level, a pair with round parentheses at the outer level can be followed only by a pair with square parentheses at the *outer* level, a square pair only by a curly one, and a curly pair only by a round one

   (b) the sequence of parenthesis pairs at the *outermost* level (those not enclosed in any other parentheses) starts by a pair with round parentheses at the outer level

   (c) a sequence of parenthesis pairs *immediately* enclosed in a pair of round parentheses (square and curly, respectively), starts by a pair with square parentheses at the outer level (curly and round, respectively)

   Sample strings *belonging* to the language:

   ( ) [ ] { } ( ) ([ ]) [{ } ([ ])] { ( ) [{ }] } ( ) [ ] { } ([{ ( ) }]) [ ]

   Sample strings *not belonging* to the language (the leftmost violation found in reading the string is underlined and briefly described, but others may follow):

erroneous string	*leftmost clause violated and description thereof*
<u>( ) { }</u> [ ] ( )	violates (a): round not followed by square; more violations of (a) follow
<u>[ ]</u> { } [ ] { ([{ }]) } ( )	violates (b): at the outermost level the string does not start by round; more violations of (a) follow
([ ]) [{ } <u>({ })</u>] { [ ] [{ }] }	violates (c): pair immediately enclosed in round does not start by square; more violations of (a) and (b) follow

   Answer the following questions:

   (a) Write a grammar that generates the language described above.

   (b) (optional) Draw the syntax tree for the valid sample string below:

   ([ ]) [{ } ([ ])] { ( ) [ ] }

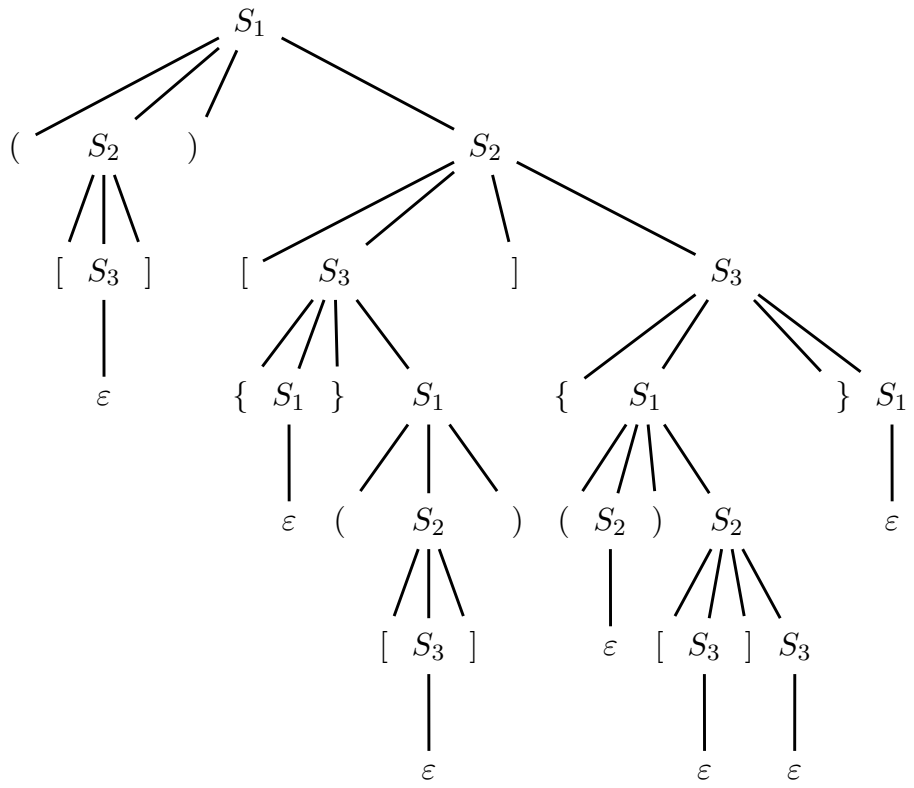
Solution

(a) The grammar required has axiom S_1 and the rules below:

$$\begin{cases} S_1 & \to & (\; S_2 \;) \; S_2 \;\; | \;\; \varepsilon \\ S_2 & \to & [\; S_3 \;] \; S_3 \;\; | \;\; \varepsilon \\ S_3 & \to & \{ \; S_1 \; \} \; S_1 \;\; | \;\; \varepsilon \end{cases}$$

Reasoning on the three clauses (a-b-c) and taking inspiration from the (counter) examples, we see that it is a sort of rotation round → square → curly → round → . . . , both in the large (concatenated pairs) and in the depth (nested pairs)

(b) Here is the syntax tree for string $\big(\; [\;] \; \big) \; \Big[\; \{ \; \} \; \big(\; [\;] \; \big) \; \Big] \; \{ \; (\;) \; [\;] \; \}$:



It is clearly visible the "rotation" of parentheses round-square-curly-round-etc, in the depth (go top-down from the root to a leaf) and in the large (this is a little less visible, but for instance inspect any two concatenated parenthesis pairs).

2. Consider part of the syntax of the *HTML* language for hypertexts. Here is how a draft *HTML* document may look like:

```
<html xmlns="www.w3.org/1999/xhtml" lang="en">
   <head>
      <title>FDTC 2013</title>
   </head>
   <body>
      <div id="main">
         <h1>Location</h1>
         <a href="/shared/UCSB_map1.png">
            <img src="/shared/UCSB_map2.png" scale=80 /img>
            <img src="/shared/pdf_file.gif" />
         </a>
      </div>
   </body>
</html>
```

Basically the document is an aggregate of records. Each record has an opening mark `<tag ... >` and a closing one `</tag>` with identical *tags* (or labels), and may contain other records between these two marks. Overall it is a typical block structure.

The document consists of exactly one record html, which must contain exactly one record head followed by one record body, and these may contain others freely concatenated and nested, tagged in any way; records head and body may not be empty.

The specifications below sufficiently clarify the details of the syntax hinted above:

- A record has the following general structure:

  ```
  <tag field_1 ... field_n> ... nested records ... </tag>
  ```

  If a record does not have any nested records, it can be (optionally) denoted as:

  ```
  <tag field_1 ... field_n /tag>

  <tag field_1 ... field_n />
  ```

  There may be one or more fields `field_i` ($n \geq 1$), or they may miss ($n = 0$).
- The fields `field_i` have the following structure, with a name and a value:

  ```
  field_name = ‘‘text’’

  field_name = number
  ```

  where `text` and `number` are a piece of text and a decimal integer.
- Suppose that the terminals tag, name, text and number are a record tag, a field name, a string and an integer, respectively, with no need of further expansion.

Write an extended grammar (*EBNF*), not ambiguous, and if necessary, reasonably complete the possibly undefined aspects of the description.

Solution

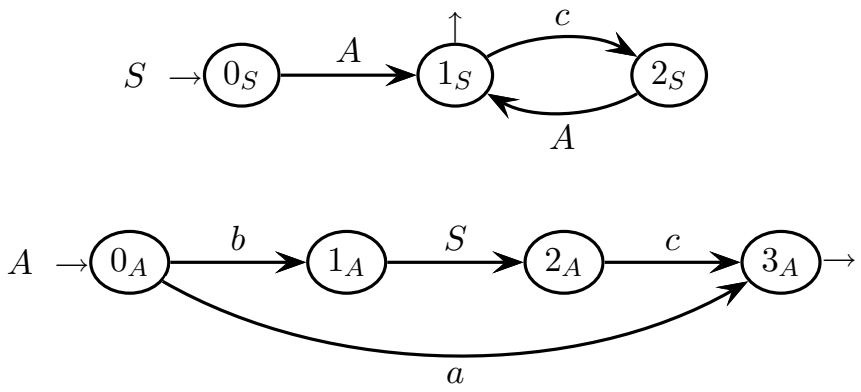
Here is a reasonable extended grammar, not ambiguous (axiom HTML):

⟨HTML⟩	→	‘<’ html ⟨FIELDS⟩ ‘>’ ⟨CONTS⟩ ‘<’ ‘/’ html ‘>’
⟨FIELDS⟩	→	(⟨FIELD⟩)*
⟨CONTS⟩	→	⟨HEAD⟩ ⟨BODY⟩
⟨FIELD⟩	→	name ‘=’ (“text” | number)
⟨HEAD⟩	→	‘<’ head ⟨FIELDS⟩ ‘>’ ⟨RECS⟩ ‘<’ ‘/’ head ‘>’
⟨BODY⟩	→	‘<’ body ⟨FIELDS⟩ ‘>’ ⟨RECS⟩ ‘<’ ‘/’ body ‘>’
⟨RECS⟩	→	(⟨REC⟩ | text)$^+$
⟨REC⟩	→	‘<’ tag ⟨FIELDS⟩ ‘>’ ⟨RECS⟩ ‘<’ ‘/’ tag ‘>’
		| ‘<’ tag ⟨FIELDS⟩ / [tag] ‘>’

The grammar consists of lists and auto-inclusive structures. Since it is designed by non-ambiguous components, reasonably it is not ambiguous either. Square brackets mean optionality. There may be several equivalent formulations.

3 Syntax Analysis and Parsing 20%

1. Consider the extended grammar (*EBNF*) that corresponds to the network of finite recursive machines shown in the figure below (axiom S):



Answer the following questions, and reasonably explain why:

(a) Determine if the grammar is $ELL(1)$, by directly verifying on the *Parser Control Flow Graph* (*PCFG*), without passing through the pilot.

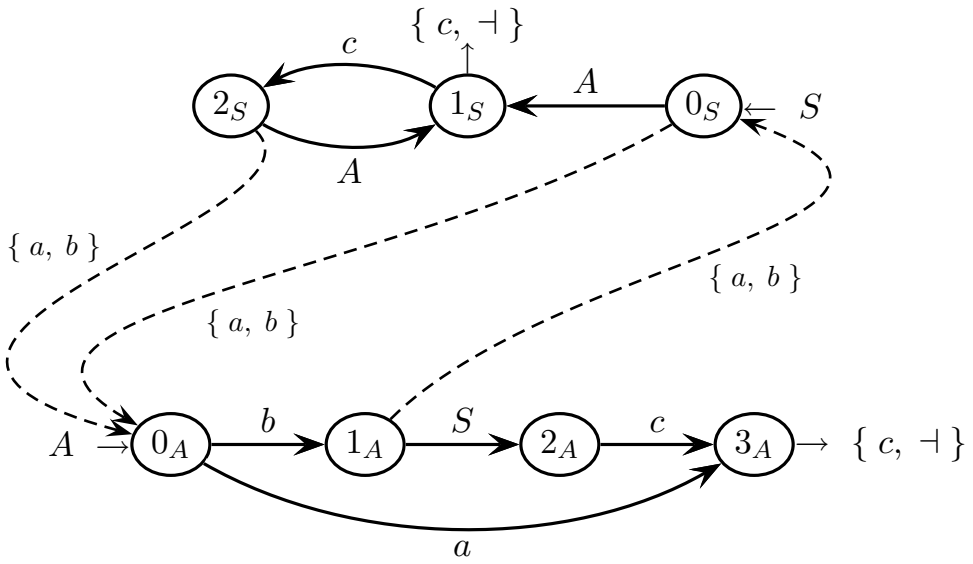
(b) Determine if the grammar is $ELR(1)$, by building the pilot.

(c) (optional) Determine if by scaling up to a prospection $k = 2$, the grammar is deterministic $ELL(2)$.

CAUTION: read the warning on the cover page.

Solution

(a) Neither nonterminal A nor S are nullable. Here is the *PCFG*, completed with the guide sets on the call arcs and exit darts:



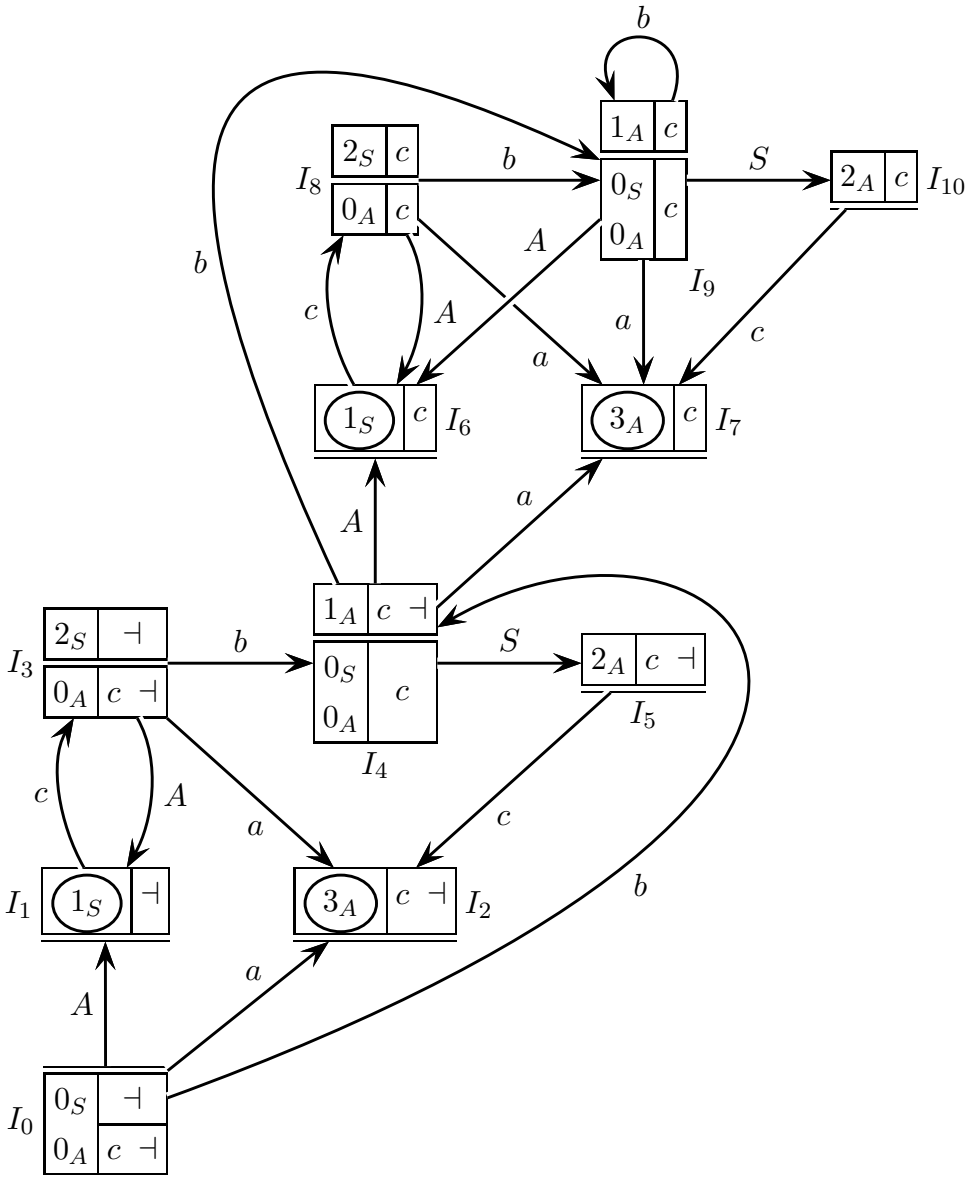
Since nonterminal A is not nullable and its machine M_A necessarily starts with a character a or b, the two call arcs directed to the initial state 0_A of M_A have a guide set $\{\ a,\ b\ \}$ and nothing else. Quite immediately, this guide set can be dragged back on the call arc directed to the initial state 0_S of the axiomatic machine M_S. This completes the guide sets on the three call arcs.

As for the guide sets on the two exit darts, that from state 0_S of M_S contains the terminator $\dashv$ because the machine M_S it belongs to, is the axiomatic one, and also contains a character c as machine M_A invokes machine M_S in the state 1_A and reads a character c soon after machine M_S has returned, so that the guide set is $\{\ c,\ \dashv\ \}$; and that from state 3_A of M_A is also $\{\ c,\ \dashv\ \}$, because both the nonterminal shift arcs that invoke machine M_A, go into state 1_S, which may be final with a character c or the terminator $\dashv$, or may continue with a shift on c.

The guide sets on the various terminal shift arcs have the same terminal as the arc label, and the figure does not display them as they are trivial.

There is a conflict in the bifurcating state 1_S between the guide set on the exit dart and that on the terminal shift arc to node 2_S, as these two sets share character c, so the grammar is not $ELL(1)$. There are not any more conflicts.

(b) Here is the complete pilot graph, with eleven m-states (initial m-state I_0):

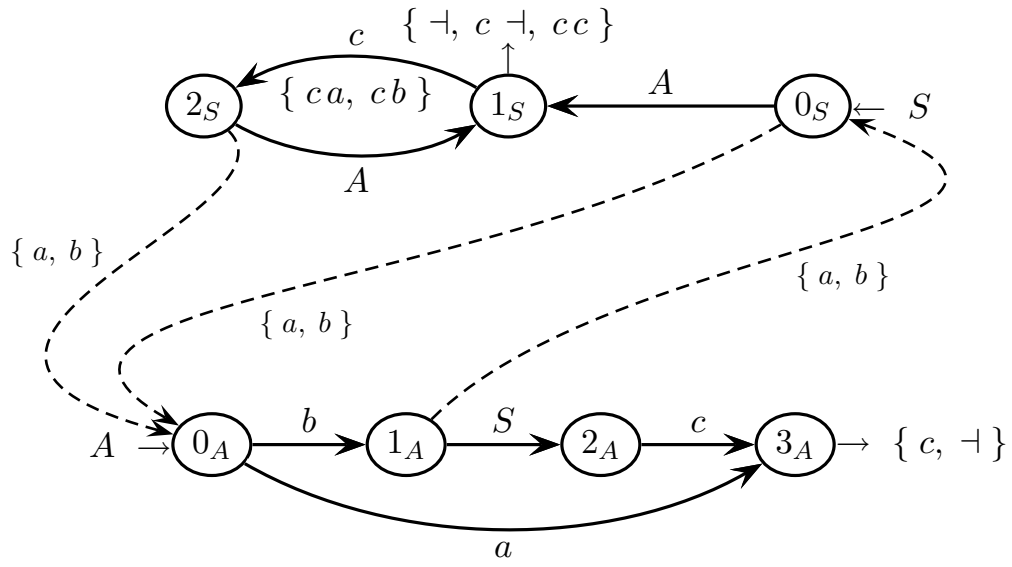


In the m-state I_6 there is a shift-reduce conflict due to character c, which can shift to m-state I_8 or enable the reduction of candidate $\langle\, 1_S\ c \,\rangle$; so the grammar is not $ELR\,(1)$.

Indirectly, we also conclude that the grammar is not $ELL\,(1)$ either, though this fact was already known from question (a). Notice that the ELL condition fails only because the grammar is not ELR, as the pilot satisfies the single transition property (STP) and the grammar does not have any left recursion.

Actually to detect the shift-reduce conflict, it would suffice to construct the pilot until the initial m-state I_0 reaches m-state I_6, and this construction at best takes only one intermediate m-state, namely I_4. There are not any more conflicts.

(c) By setting prospection $k = 2$, we have to reconsider the guide sets on the conflicting bifurcation states of the *PCFG*; in this case, only on state 1_S; the other guide sets are unchanged. Here is the result, and below is the explanation:



We recompute the guide set on the shift arc $1_S \to 2_S$, and we get set $\{\ c\,a,\ c\,b\ \}$ since c is followed by A, which is not nullable and has to start by a or b.

Similarly, we recompute the guide set on the exit dart from state 1_S, and we get set $\{\ \dashv,\ c\ \dashv,\ c\,c\ \}$ since S may be followed by $\dashv$, by c in the machine A and then by $\dashv$, or by c in the machine A and then one more c at the exit from machine A.

Then we see that these two guide sets do not overlap any more, as the former common element, character c, is now expanded to $c\,a$ and $c\,b$ in one set, and to $c\,c$ in the other set (the elements $\dashv$ and $c\ \dashv$ correspond to the end of the string and do not matter here). So we conclude that the grammar is $ELL(2)$.

We take the opportunity to comment a little more about lengthening the prospection, just for doing some useful exercise. Since we have concluded before that the grammar is $ELL(2)$, intuitively it should be $ELR(2)$ as well[1], despite a formal presentation of the ELR theory for $k > 1$ was not given. However, we can have a closer look at the conflicting m-state I_6 in the pilot, and simulate what happens if we want to shift on two terminals, which is a way for understanding what parsing actions are possible within a prospection window of two characters. Two situations may occur, namely:

- by a shift on terminal c, from m-state I_6 we go to m-state I_8, then we have to shift[2] on terminal a or b; we can distinguish either these terminal shifts by means of a prospection of one more character after c (so we need a prospection $k = 2$)
- the latter case fires a longer sequence of events, as follows:
  - first by reducing[3] candidate $\langle\, 1_S\ c \,\rangle$, from m-state I_6 we go back to the m-state I_4, for it contains the initial candidate $\langle\, 0_S\ c \,\rangle$
  - then we must shift on nonterminal S and go to m-state I_5, which does not have any reduction candidates
  - then we have to shift[4] on terminal c and go to m-state I_2, which is a reduction for candidate $\langle\, 3_A\ \{\ c\ \dashv \}\,\rangle$
  - then we have to reduce[5] candidate $\langle\, 3_A\ \{\ c\ \dashv \}\,\rangle$, and from m-state I_2 we go back to the m-state I_3, for it contains the initial candidate $\langle\, 0_A\ \{\ c\ \dashv \}\,\rangle$
  - then we must shift on nonterminal A and go to m-state I_1, which is a reduction for candidate $\langle\, 1_S\ \dashv\rangle$
  - finally, two sub-cases are possible:
    - ***shift*** if we are not at the string end, then we have to shift on terminal c
    - ***reduce*** otherwise (i.e., string end) we have to reduce candidate $\langle\, 1_S\ \dashv\rangle$

  therefore we can distinguish these last terminal *shift* or *reduction*, by means of a prospection of one more character after the first character c, already shifted before when going to m-state I_2 (so we need a prospection $k = 2$ again)

The conclusion is that a prospection window of size $k = 2$ helps us solve the shift-reduce conflict in the m-state I_6, as follows:

- a prospection pair $c\,a$ or $c\,b$ is for shifting from I_6 to I_8, reading the c
- while a prospection pair $c\,c$ or $c\ \dashv$ is for reducing candidate $\langle\, 1_S\ c\,\rangle$, reading the first c when the shift $I_5 \rightarrow I_2$ takes place subsequently

This wider prospection is only needed for the m-state I_6; all the other m-states do well with a narrower prospection $k = 1$. In this intuitive sense, the grammar is $ELR(2)$.

[1] Remember that the LL analysis concept is a restriction of the LR one, since it implies to make parsing decisions with less information.

[2] A shift on nonterminal A might occur only after a reduction to A, which here is out of place as the currently active machine is M_S.

[3] Since machine M_S contains a loop, the reduction handle to be popped from the stack may be unbounded and contain a long series of stack symbols, namely I_8 and I_6, which are the looping ones.

[4] This shift corresponds to the look-ahead c of the reduction candidate $\langle\, 1_S\ c\,\rangle$.

[5] This time the reduction handle to be popped is of bounded length, for machine M_A is loop-free.

4 Translation and Semantic Analysis 20%

1. One wishes one transformed a program written in C or Java style, which uses curly brackets as delimiters, into an equivalent one in Pascal or Ada style, which uses the keywords *begin*, *end*, *then*, *else* and *endif*. The source and image languages, and how to transform the former into the latter, are illustrated by the example below:

```
{                                   |  begin
    if c1 {                         |      if c1 then
        s1 ; s2 ; s3 ;              |          s1 ; s2 ; s3
    } ; else {                      |      else
        s4 ; s5 ;                   |          s4 ; s5 ;
        if c2 {                     |          if c2 then
            s6 ;                    |              s6
        } ;                         |          endif
        s7 ;                        |          s7
    } ;                             |      endif
    while c3 do {                   |      while c3 do begin
        s8 ; s9 ;                   |          s8 ; s9
    } ;                             |      end
} ;                                 |  end
```

The source and image languages are defined by the *EBNF* grammars G_1 and G_2 below, which must be changed into not extended (*BNF*) to write the transduction.

$$G_1 \begin{cases} \langle \mathsf{prog1} \rangle & \rightarrow & \text{`\{'} \ (\ \langle \mathsf{stat1} \rangle \ ;\ )^+ \ \text{`\}'} \ ; \\ \langle \mathsf{stat1} \rangle & \rightarrow & \textbf{if}\ c\ \text{`\{'}\ (\langle \mathsf{stat1} \rangle \ ;\ )^+ \ \text{`\}'}\ [\ ;\ \textbf{else}\ \text{`\{'}\ (\ \langle \mathsf{stat1} \rangle \ ;\ )^+ \ \text{`\}'}\ ] \\ \langle \mathsf{stat1} \rangle & \rightarrow & \textbf{while}\ c\ \textbf{do}\ \text{`\{'}\ (\ \langle \mathsf{stat1} \rangle \ ;\ )^+ \ \text{`\}'} \\ \langle \mathsf{stat1} \rangle & \rightarrow & s \qquad \text{- - any simple statement} \end{cases}$$

$$G_2 \begin{cases} \langle \mathsf{prog2} \rangle & \rightarrow & \textbf{begin}\ (\ \langle \mathsf{stat2} \rangle \ ;\ )^*\ \langle \mathsf{stat2} \rangle\ \textbf{end} \\ \langle \mathsf{stat2} \rangle & \rightarrow & \textbf{if}\ c\ \textbf{then}\ (\ \langle \mathsf{stat2} \rangle \ ;\ )^*\ \langle \mathsf{stat2} \rangle\ [\ \textbf{else}\ (\ \langle \mathsf{stat2} \rangle \ ;\ )^*\ \langle \mathsf{stat2} \rangle\ ]\ \textbf{endif} \\ \langle \mathsf{stat2} \rangle & \rightarrow & \textbf{while}\ c\ \textbf{do begin}\ (\ \langle \mathsf{stat2} \rangle \ ;\ )^*\ \langle \mathsf{stat2} \rangle\ \textbf{end} \\ \langle \mathsf{stat2} \rangle & \rightarrow & s \qquad \text{- - any simple statement} \end{cases}$$

The axioms of G_1 and G_2 are prog1 and prog2. Caution: the semicolon ";" is missing in the image language, if it is immediately followed by a keyword *end* or *endif*.

Answer the following questions:

(a) Write a syntax transduction scheme (no attributes) for the translation above.

(b) (optional) Draw the translation syntax tree of the sample source string below:

$$\Big\{\ \textbf{if}\ c_1\ \{\ s_1\ ;\ \}\ ;\ \textbf{else}\ \{\ \textbf{if}\ c_2\ \{\ \textbf{while}\ c_3\ \textbf{do}\ \{\ s_2\ ;\ \}\ ;\ \}\ ;\ \}\ ;\ s_3\ ;\ \Big\}\ ;$$

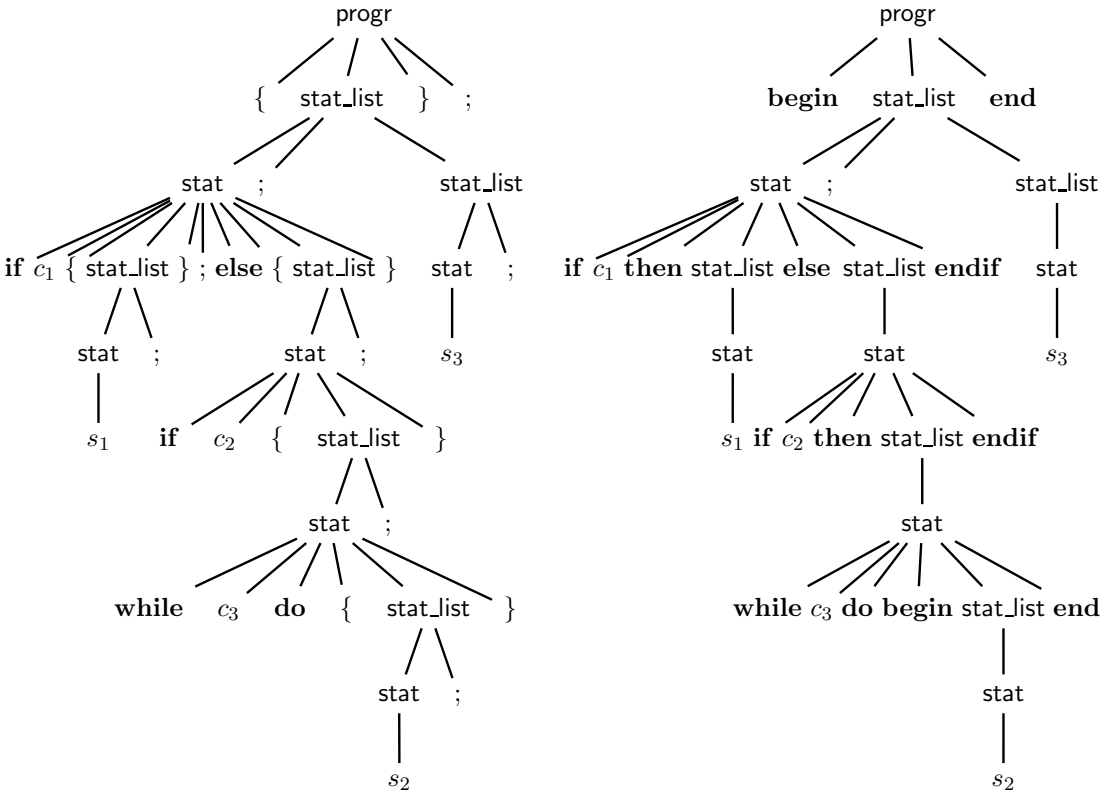
Solution

(a) Here is the syntax scheme, in the non-extended form (*BNF*) as required:

$\langle \text{progr} \rangle \to \{\ \langle \text{stat\_list} \rangle\ \}\ ;$	$\langle \text{progr} \rangle \to \textbf{begin}\ \langle \text{stat\_list} \rangle\ \textbf{end}$
$\langle \text{stat\_list} \rangle \to \langle \text{stat} \rangle\ ;\ \langle \text{stat\_list} \rangle$	$\langle \text{stat\_list} \rangle \to \langle \text{stat} \rangle\ ;\ \langle \text{stat\_list} \rangle$
$\langle \text{stat\_list} \rangle \to \langle \text{stat} \rangle\ ;$	$\langle \text{stat\_list} \rangle \to \langle \text{stat} \rangle$
$\langle \text{stat} \rangle \to s$	$\langle \text{stat} \rangle \to s$
$\langle \text{stat} \rangle \to \textbf{if}\ c\ \{\ \langle \text{stat\_list} \rangle\ \}$	$\langle \text{stat} \rangle \to \textbf{if}\ c\ \textbf{then}\ \langle \text{stat\_list} \rangle\ \textbf{endif}$
$\langle \text{stat} \rangle \to \textbf{if}\ c\ \{\ \langle \text{stat\_list} \rangle\ \}\ ;$ $\textbf{else}\ \{\ \langle \text{stat\_list} \rangle\ \}$	$\langle \text{stat} \rangle \to \textbf{if}\ c\ \textbf{then}\ \langle \text{stat\_list} \rangle$ $\textbf{else}\ \langle \text{stat\_list} \rangle\ \textbf{endif}$
$\langle \text{stat} \rangle \to \textbf{while}\ c\ \textbf{do}$ $\{\ \langle \text{stat\_list} \rangle\ \}$	$\langle \text{stat} \rangle \to \textbf{while}\ c\ \textbf{do}$ $\textbf{begin}\ \langle \text{stat\_list} \rangle\ \textbf{end}$

Notice that the statement list is rendered by recursion and that in the destination grammar the semicolon "**;**" is absent before the other closing delimiters.

(b) Here are the source and image syntax trees of the proposed phrase:

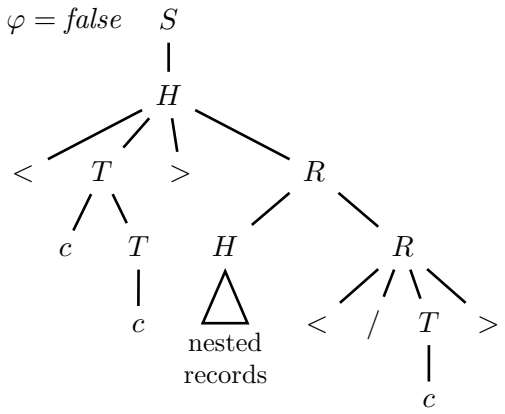


The translation phrase is the following:

begin if c_1 **then** s_1 **else if** c_2 **then while** c_3 **do**
begin s_2 **end endif endif**; s_3 **end**

2. Consider the syntax below, in the not extended form (*BNF*), which generates a part (very simplified) of the *HTML* language for hypertexts (axiom S).

$$
\begin{array}{lllll}
1: & S & \rightarrow & H & \\
2: & H & \rightarrow & < \ T \ > & R \\
3: & R & \rightarrow & H \ R & \\
4: & R & \rightarrow & < \ / \ T \ > & \\
5: & T & \rightarrow & c \ T & \\
6: & T & \rightarrow & c &
\end{array}
$$



Aside the syntax support, there is a sample syntax tree (partial). Those who want so, may see the text of ex. 2.2 for the few notions about *HTML* that suffice for here.

The syntax above does not ensure that the opening $< \ T \ >$ and closing $< \ / \ T \ >$ tags of a record match. One wishes one added such a control by means of attributes.

Answer the following questions:

(a) Write the semantic functions to compute a boolean attribute φ in the tree root S, which is *true* if and only if the opening and closing tags of each record match (see the sample tree above). Use the attributes listed below (do not add more attributes), and specify the type of the *tag* and *carry* attributes:

name	*type*	*symbol*	*domain*	*meaning*
φ	left	H, R, S	boolean	*true* iff the opening and closing tags of each record coincide
tag		T	string	opening or closing tag
carry		R	string	tag transported through the tree for comparing it with the corresponding tag

(b) Discuss if the attribute grammar found at point (a), is one sweep. Since the syntax is $LL(2)$, discuss if the semantic analyzer based on such a grammar can be integrated with the syntactic $LL(2)$ one.

(c) (optional) One wishes the semantic analyzer emitted an error message as soon as it has finished to read the closing tag of a record and finds out the tag does not match the opening one (of the same record). Can it be done and integrated with the syntactic $LL(2)$ analyzer? (explain why) If it is impossible, argue how to change the semantic functions (not the syntax) so as to make it possible.

Solution

(a) Purely synthesized version. Both attributes *tag* and *carry* are left.

#	*syntax*	*semantic*
1:	$S_0 \rightarrow H_1$	$\varphi_0 := \varphi_1$

2:	$H_0 \rightarrow < T_1 > R_2$	**if** $tag_1 = carry_2$ **then**
		$\varphi_0 := \varphi_2$ - - watch it !
		else
		$\varphi_0 := false$
		end if

3:	$R_0 \rightarrow H_1 R_2$	$\varphi_0 := \varphi_1 \wedge \varphi_2$ $carry_0 := carry_2$
4:	$R_0 \rightarrow < / T_1 >$	$\varphi_0 := true$ $carry_0 := tag_1$
5:	$T_0 \rightarrow c\, T_1$	$tag_0 := c \cdot tag_1$
6:	$T_0 \rightarrow c$	$tag_0 := c$

The basic idea is simple: at rule 4 compute the closing tag by attribute *tag*, transport it *upwards* to rule 2 by attribute *carry*, and in the rule 2 compare it with the opening tag also computed by *tag*; if the two tags do not match, set attribute φ to false; otherwise set it to the value of the attribute φ coming from the subtree of the nested records (for it is the nested records that will decide).

At the rule 3 branching the nested records, the two instances of attribute φ coming up from the nested records and the current one, are logically conjuncted. Eventually attribute φ reaches the root, after collecting all the tag checks. The rest of the solution is quite obvious and does not need any specific discussion.

Version with inheritance. Attribute *tag* is left and attribute *carry* is right.

#	*syntax*	*semantic*
1:	$S_0 \rightarrow H_1$	$\varphi_0 := \varphi_1$
2:	$H_0 \rightarrow < T_1 > R_2$	$carry_2 := tag_1$ $\varphi_0 := \varphi_2$
3:	$R_0 \rightarrow H_1\ R_2$	$carry_2 := carry_0$ $\varphi_0 := \varphi_1 \wedge \varphi_2$

4:	$R_0 \rightarrow < / T_1 >$	**if** $carry_0 = tag_1$ **then**
		$\varphi_0 := true$
		else
		$\varphi_0 := false$
		end if

5:	$T_0 \rightarrow c\,T_1$	$tag_0 := c \cdot tag_1$
6:	$T_0 \rightarrow c$	$tag_0 := c$

The basic idea is not too different from the previous one, but with introducing inheritance: at rule 2 compute the opening tag by attribute *tag*, transport it *donwards* to rule 4 by attribute *carry*, and in the rule 4 compare it with the closing tag also computed by *tag*; if the two tags match, set attribute φ to true; otherwise set it to false. So now attribute *carry* is inherited (right).

As before, at the rule 3 branching the nested records, the two instances of attribute φ coming up from the nested records and the current one, are logically conjuncted. Eventually attribute φ reaches the root, after collecting all the tag checks. The rest of the solution is also quite obvious.

(b) Both versions are correct, in the sense that there are not any circular dependences among the attributes. Since the first version is purely synthesized, it is surely one-sweep. One can easily check that also the second version, that with inheritance, satisfies the one-sweep condition. So it is computable in one sweep, by following the left to right ordering of the symbols in the right parts of the rules (it is not necessary to permute the child nodes).

Since the first version is purely synthesized and so one-sweep, and moreover its syntax is deterministic $LL(2)$, it can be surely integrated with the syntactic $LL(2)$ analyzer. Also the second version, with inheritance, can be integrated with the syntactic $LL(2)$ analyzer, because it is one-sweep as well, and the visiting order of the nodes in the rules 2 and 3 is left to right (L condition); one can easily verify that it is so, by drawing the successor graph.

(c) In order to add a diagnostic of "tag mismatch" to the semantic analyzer, as soon as the analysis has finished to read a closing tag and discovers that it does not match the opening one, we clearly have to resort to the version with inheritance. It is sufficient to modify the semantic associated with rule 4, this way:

#	*syntax*	*semantic*
...		...

4:	$R_0 \rightarrow < / T_1 >$	**if** $carry_0 = tag_1$ **then**
		  $\varphi_0 := true$
		else
		  $\varphi_0 := false$
		  print ("tags", $carry_0$, tag_1, "do not match")
		end if

...		...

The other rules are unchanged. The semantic analyzer with diagnostics can be integrated with the syntactic $LL(2)$ analyzer as well, as explained before.